

Deep Unsupervised Learning

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

August 9, 2026

1. Motivation
2. What is Deep Unsupervised Learning?
3. Learning Outcomes
4. Components / Supervised vs Unsupervised
5. Applications
6. Challenges
7. Types: Clustering, Dimensionality Reduction, Anomaly Detection
8. Clustering: K-means, Hierarchical, DBSCAN
9. Dimensionality Reduction: PCA, t-SNE, UMAP
10. References

Supervised learning is effective, but it has some limitations:

- ▶ Needs a lot of labeled data to work well.
- ▶ Getting labels can be difficult or costly.
- ▶ Depends on experts to label the data.
- ▶ Not practical for many real-world situations where labeling is hard or impossible.

- ▶ Humans are good at:
 - Learning patterns from experience
 - Discovering hidden structures
- ▶ But:
 - They are slow and limited
 - Not always available
 - Can't always explain what they observe

- ▶ What if machines could learn to:
 - Find hidden patterns
 - Understand structures in data
 - Group and compare similar things

Without any human-provided labels?

- ▶ Learns **without labels**
- ▶ Discovers **structure within the data**
- ▶ Mimics how humans learn from **observation**
- ▶ Useful for:
 - Clustering
 - Dimensionality reduction
 - Representation learning

- ▶ Simple models can't handle:
 - High variability in real-world data
 - Complex abstract features
- ▶ We need:
 - Neural networks
 - With many layers → **“Deep”**
 - To learn hierarchical patterns

- ▶ Discover hidden patterns
- ▶ Learn rich representations
- ▶ Group and distinguish data points
- ▶ Match similar structures

Deep Unsupervised Learning aims to discover **hidden patterns** and **structures** in raw data using deep neural networks, **without any human-provided labels**.

It focuses on **capturing rich patterns in raw data with deep networks in a label-free way**.

- ▶ **Learns from observation:** mimics how humans learn from experience
- ▶ **Discovers structure:** finds groups, similarities, and representations
- ▶ **Scales to complex data:** uses deep models to capture abstract features
- ▶ **Generative Models:** recreate raw data distribution
- ▶ **Self-supervised Learning:** “puzzle” tasks that require semantic understanding

Why do we care?



Geoffrey Hinton
(in his 2014 AMA on Reddit)

“The brain has about 10^{14} synapses and we only live for about 10^9 seconds. So we have a lot more parameters than data. This motivates the idea that we must do a lot of unsupervised learning since the perceptual input (including proprioception) is the only place we can get 10^5 dimensions of constraint per second.”



Yann LeCun

Need tremendous amount of information to build machines that have common sense and generalize well.

[LeCun, "Predictive Learning" keynote, NeurIPS 2016]

■ "Pure" Reinforcement Learning (cherry)

- ▶ The machine predicts a scalar reward given once in a while.
- ▶ **A few bits for some samples**

■ Supervised Learning (icing)

- ▶ The machine predicts a category or a few numbers for each input
- ▶ Predicting human-supplied data
- ▶ **10→10,000 bits per sample**

■ Unsupervised/Predictive Learning (cake)

- ▶ The machine predicts any part of its input for any observed part.
- ▶ Predicts future frames in videos
- ▶ **Millions of bits per sample**

■ (Yes, I know, this picture is slightly offensive to RL folks. But I'll make it up)



- ▶ “Ideal Intelligence” is all about compression (finding all patterns)
- ▶ Finding all patterns = short description of raw data (*low Kolmogorov Complexity*)
- ▶ Shortest code-length = optimal inference (*Solomonoff Induction*)
- ▶ Extensible to optimal action-making agents (*AIXI*)

Transfer Learning via Compression:

- ▶ Assume we pretrain unsupervised on Data Distribution D_1 and then finetune on Data Distribution D_2
- ▶ If D_1 and D_2 are related, compressing D_2 conditioned on D_1 should be more efficient than compressing D_2 outright
- ▶ **Hence:** pretraining on D_1 should aid faster learning of D_2

By the end of this session, you will be able to:

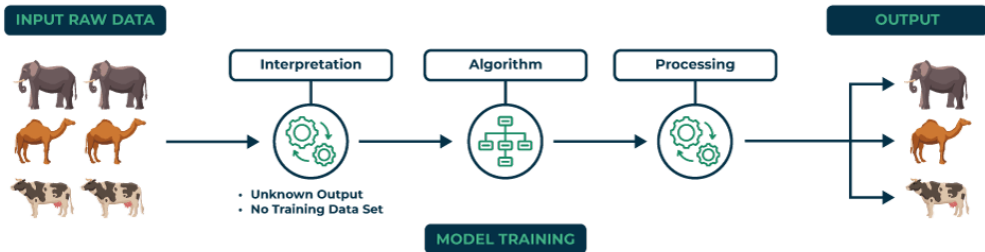
1. **Define** deep unsupervised learning and **differentiate** it from supervised approaches.
2. **Understand** the importance and real-world applications of unsupervised learning in:
 - Natural language processing
 - Computer vision
 - Fraud detection
3. **Identify** key challenges in unsupervised learning:
 - Model evaluation
 - Lack of labeled ground truth

4. **Classify** main types of unsupervised learning tasks:
 - Clustering
 - Dimensionality reduction
 - Anomaly detection
5. **Compare** the main clustering algorithms and the assumptions each one makes:
 - K-means
 - Hierarchical clustering
 - DBSCAN
6. **Explain** core dimensionality reduction techniques:
 - Principal Component Analysis (PCA)
 - t-Distributed Stochastic Neighbor Embedding (t-SNE)
 - Uniform Manifold Approximation and Projection (UMAP)

- ▶ **Data:** Unlabeled data, e.g., images, text, or sensor readings.
- ▶ **Model:** A mathematical representation of the data, e.g., a mixture model or a neural network.
- ▶ **Objective function:** A measure of how well the model fits the data, e.g., likelihood or reconstruction error.
- ▶ **Optimization algorithm:** An algorithm to minimize the objective function, e.g., gradient descent or expectation-maximization.
- ▶ **Evaluation metrics:** Measures to assess the quality of the learned model, e.g., silhouette score or clustering accuracy.

Applications: Use cases for unsupervised learning, e.g., clustering, dimensionality reduction, or anomaly detection.

Unsupervised Learning



Source: [geeksforgeeks.org](https://www.geeksforgeeks.org/)

Aspect	Supervised Learning	Unsupervised Learning
Objective	Learn a function f from labeled input-output pairs.	Discover structure or representations in unlabeled data.
Evaluation	Accuracy, precision/recall on held-out labels.	Clustering validity indices (e.g. silhouette), reconstruction error.
Cost	Methods range from $\mathcal{O}(n)$ to $\mathcal{O}(n^3)$ per fit.	k-means $\mathcal{O}(nkd)$, hierarchical $\mathcal{O}(n^2)$, PCA $\mathcal{O}(nd^2)$.
Labels/Clusters	Fixed, known set of classes.	Number of clusters unknown; must be chosen or inferred.
Output	Classifier or regressor for new inputs.	Cluster assignments, embeddings, density models, or generative samples.

Table 1: Key differences between Supervised and Unsupervised Learning

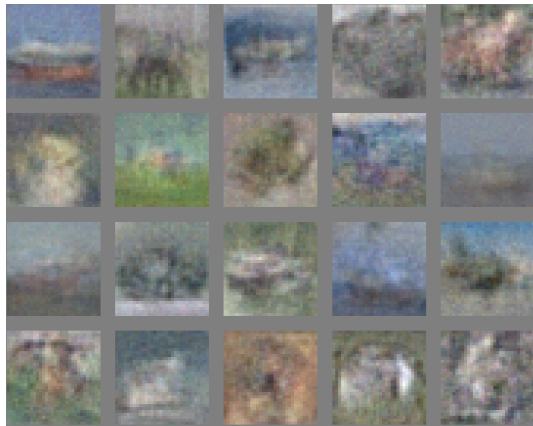
Deep Unsupervised Learning is already used across several areas:

- ▶ **Generate Novel Data:** Create new data samples similar to the training set.
- ▶ **Conditional Synthesis Technology:** Enable advanced applications such as WaveNet for audio generation and GAN-based models like pix2pix for image-to-image translation.
- ▶ **Compression:** Learn compact representations for efficient data storage and transmission.
- ▶ **Improve Downstream Tasks:** Enhance performance of supervised tasks through unsupervised or self-supervised pre-training.
- ▶ **Production-Level Impact:** Power real-world systems, e.g., Google Search improvements with BERT pre-training.
- ▶ **Flexible Building Blocks:** Provide reusable components for larger machine learning pipelines.



Figure 1: [Deep Belief Nets, Hinton, Osindero, Teh, 2006]

Application: Generate Images (cont.)



[GAN, Goodfellow et al. 2014]

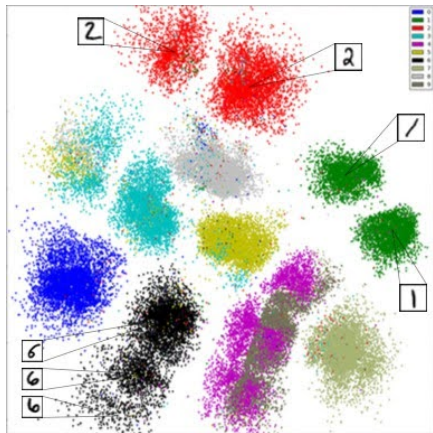


Figure 2: Unsupervised learning can discover structure in digits without any labels.

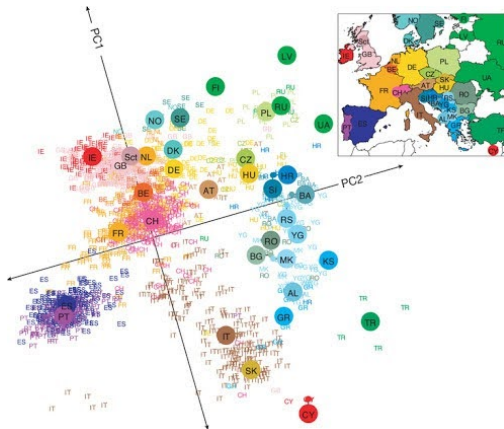


Figure 3: Dimensionality reduction applied to DNA reveals the geography of European countries.
[Novembre et al., “Genes mirror geography within Europe”, Nature 456, 2008]

“a masterful oil painting a persian exotic cat discovering their astounding crypto losses while checking their phone”



[DALL·E 2, OpenAI, 2022]

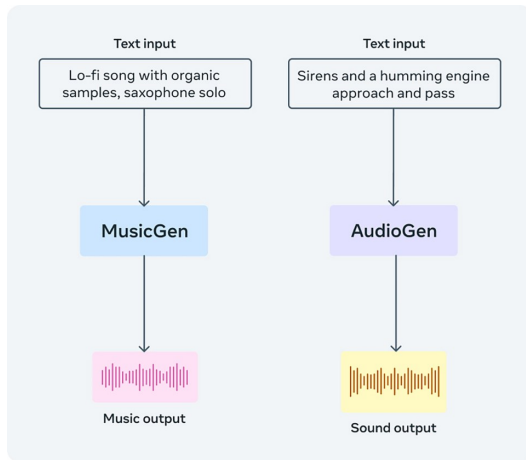


Figure 4: [AudioCraft, Copet et al., 2023]

Application: Text to Speech Generation

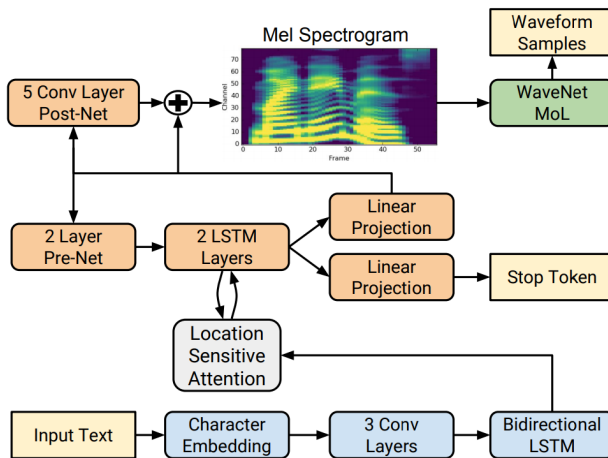


Figure 5: [Tacotron2, Shen et al., 2018]

For $\bigoplus_{n=1, \dots, m} \mathcal{L}_{m,n}$ where $\mathcal{L}_{m,n} = 0$, hence we can find a closed subset $\mathcal{H}$ in $\mathcal{H}$ and any sets $\mathcal{F}$ on X , U is a closed immersion of S , then $U \rightarrow T$ is a separated algebraic space.

Proof. Proof of (1). It also start we get

$$S = \text{Spec}(R) = U \times_X U \times_X U$$

and the comparably in the fibre product covering we have to prove the lemma generated by $\prod Z \times_U U \rightarrow V$. Consider the maps M along the set of points Sch_{fppf} and $U \rightarrow U$ is the fibre category of S in U in Section, ?? and the fact that any U affine, see Morphisms, Lemma ?? . Hence we obtain a scheme S and any open subset $W \subset U$ in $\text{Sh}(G)$ such that $\text{Spec}(R') \rightarrow S$ is smooth or an

$$U = \bigcup U_i \times_{S_i} U_i$$

which has a nonzero morphism we may assume that f_i is of finite presentation over S . We claim that $\mathcal{O}_{X,x}$ is a scheme where $x, x', s'' \in S'$ such that $\mathcal{O}_{X,x'} \rightarrow \mathcal{O}'_{X',x'}$ is separated. By Algebra, Lemma ?? we can define a map of complexes $\text{GL}_{S'}(x'/S'')$ and we win. $\square$

To prove study we see that $\mathcal{F}|_U$ is a covering of X' , and $\mathcal{T}_i$ is an object of $\mathcal{F}_{X/S}$ for $i > 0$ and $\mathcal{F}_p$ exists and let $\mathcal{F}_i$ be a presheaf of $\mathcal{O}_X$ -modules on $\mathcal{C}$ as a $\mathcal{F}$ -module. In particular $\mathcal{F} = U/\mathcal{F}$ we have to show that

$$\tilde{M}^\bullet = I^\bullet \otimes_{\text{Spec}(k)} \mathcal{O}_{S,s} - i_X^{-1} \mathcal{F}$$

is a unique morphism of algebraic stacks. Note that

$$\text{Arrows} = (\text{Sch}/S)_{fppf}^{\text{opp}}, (\text{Sch}/S)_{fppf}$$

and

$$V = \Gamma(S, \mathcal{O}) \rightarrow (\mathcal{U}, \text{Spec}(A))$$

is an open subset of X . Thus U is affine. This is a continuous map of X is the inverse, the groupoid scheme S .

Proof. See discussion of sheaves of sets. $\square$

The result to prove any open covering follows from the less of Example ?? . It may replace S by $X_{\text{spaces, étale}}$ which gives an open subspace of X and T equal to S_{Zar} , see Descent, Lemma ?? . Namely, by Lemma ?? we see that R is geometrically regular over S .

Lemma 0.1. Assume (3) and (3) by the construction in the description.

Suppose $X = \lim |X|$ (by the formal open covering X and a single map $\text{Proj}_X(\mathcal{A}) = \text{Spec}(B)$ over U compatible with the complex

$$\text{Set}(\mathcal{A}) = \Gamma(X, \mathcal{O}_{X, \mathcal{O}_X}).$$

When in this case of to show that $\mathcal{Q} \rightarrow \mathcal{C}_{Z/X}$ is stable under the following result in the second conditions of (1), and (3). This finishes the proof. By Definition ?? (without element is when the closed subschemes are catenary. If T is surjective we may assume that T is connected with residue fields of S . Moreover there exists a closed subspace $Z \subset X$ of X where U in X' is proper (some defining as a closed subset of the uniqueness it suffices to check the fact that the following theorem

(1) f is locally of finite type. Since $S = \text{Spec}(R)$ and $Y = \text{Spec}(R)$.

Proof. This is form all sheaves of sheaves on X . But given a scheme U and a surjective étale morphism $U \rightarrow X$. Let $U \cap U = \prod_{i=1, \dots, n} U_i$ be the scheme X over S at the schemes $X_i \rightarrow X$ and $U = \lim_i X_i$. $\square$

The following lemma surjective restocomposes of this implies that $\mathcal{F}_{x_0} = \mathcal{F}_{x_0} = \mathcal{F}_{X, \dots, 0}$.

Lemma 0.2. Let X be a locally Noetherian scheme over S , $E = \mathcal{F}_{X/S}$. Set $\mathcal{I} = \mathcal{I}_1 \subset \mathcal{I}_n$. Since $\mathcal{I}^n \subset \mathcal{I}^n$ are nonzero over $i_0 \leq \mathfrak{p}$ is a subset of $\mathcal{I}_{n,0} \circ \bar{A}_2$ works.

Lemma 0.3. In Situation ?? . Hence we may assume $\mathfrak{q}' = 0$.

Proof. We will use the property we see that $\mathfrak{p}$ is the next functor (??). On the other hand, by Lemma ?? we see that

$$D(\mathcal{O}_{X'}) = \mathcal{O}_X(D)$$

where K is an F -algebra where δ_{n+1} is a scheme over S . $\square$

Figure 6: [Char-RNN, Karpathy, 2015]

Prompt [human written]: A train carriage containing controlled nuclear materials was stolen in Cincinnati today. Its whereabouts are unknown.

GPT-2: The incident occurred on the downtown train line, which runs from Covington and Ashland stations. In an email to Ohio news outlets, the U.S. Department of Energy said it is working with the Federal Railroad Administration to find the thief. 'The theft of this nuclear material will have significant negative consequences on public and environmental health, our workforce and the economy of our nation,' said Tom Hicks, the U.S. Energy Secretary, in a statement. 'Our top priority is to secure the theft and ensure it doesn't happen again.' The stolen material was taken from the University of Cincinnati's Research Triangle Park nuclear research site, according to a news release from Department officials. The Nuclear Regulatory Commission did not immediately release any information. According to the release, the U.S. Department of Energy's Office of Nuclear Material Safety and Security is leading that team's investigation. 'The safety of people, the environment and the nation's nuclear stockpile is our highest priority,' Hicks said. 'We will get to the bottom of this and make no excuses.'

[OpenAI's GPT2 – Radford, Wu, Child, Luan, Amodei, Sutskever, 2019]

Why is unsupervised learning challenging?

- ▶ **Exploratory data analysis:** Unsupervised learning is often used for exploratory data analysis, where the goal is to discover patterns or structures in the data without any prior knowledge of the labels.
- ▶ **Difficult to assess performance:** Evaluating the performance of unsupervised learning algorithms can be challenging, as there are no ground truth labels to compare against (“right answer” unknown).
- ▶ **Sensitivity to noise:** Unsupervised learning algorithms can be sensitive to noise and outliers in the data, which can lead to misleading results.
- ▶ **Curse of dimensionality:** As the number of features increases, the data becomes sparse, making it difficult to find meaningful patterns.
- ▶ **Lack of labeled data:** There is no supervision signal to guide the learning process toward task-relevant structure.

- ▶ **Interpretability issues:** The patterns and representations discovered can be hard to interpret or validate.

► Cluster Analysis:

- For identifying homogeneous subgroups of samples.
- **Examples:** K-means, hierarchical clustering, DBSCAN.

► Dimensionality Reduction:

- For finding a low-dimensional representation to characterize and visualize the data.
- Reducing the number of features in a dataset while preserving important information.
- **Examples:** PCA, t-SNE, UMAP.

► Anomaly Detection:

- **Finding outliers in the dataset:** Identifying unusual (rare items, events, or observations) data points that do not conform to expected patterns.
- **Examples:** Isolation Forest, One-Class SVM, Autoencoders.

A set of methods for finding subgroups within the dataset.

- ▶ Observations should share common characteristics within the same group, but differ across groups.
- ▶ Groupings are determined from attributes of the data itself, which is what differs from classification.

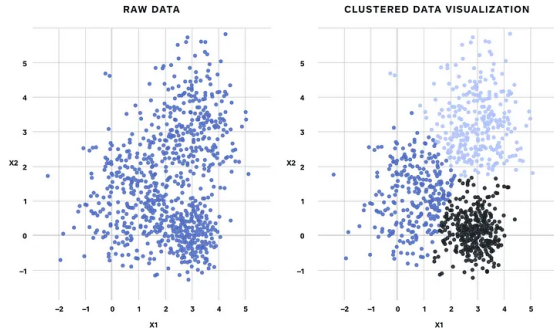


Figure 7: Taking a 2 dimensional dataset and separating it into 3 distinct clusters. [Source]

Input: Dataset $D = \{x_1, x_2, \dots, x_n\}$, number of clusters k

Output: Cluster assignments for each data point

Initialization: Randomly initialize k cluster centroids or seeds;

repeat

Assignment Step: Assign each data point x_i to the nearest cluster based on a distance metric;

Update Step: Recompute cluster centroids using current assignments;

until *convergence or maximum iterations reached*;

return Final cluster assignments;

Algorithm: Centroid-Based (K-means-style) Clustering

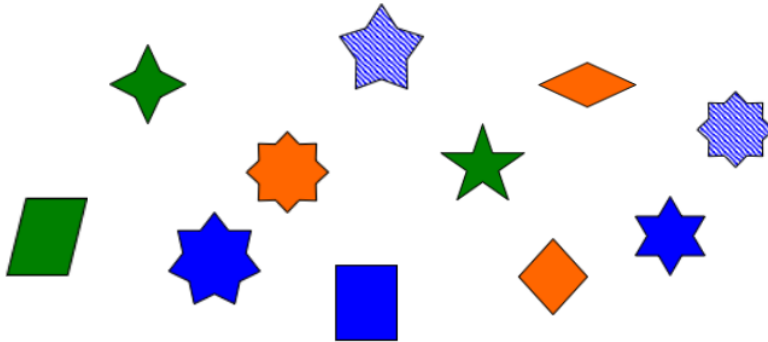


Figure 8: Sample data points. [Source: CME 250, S. Wang, Stanford]

Classification

- ▶ Labels available
- ▶ Assigning to known classes
- ▶ Supervised

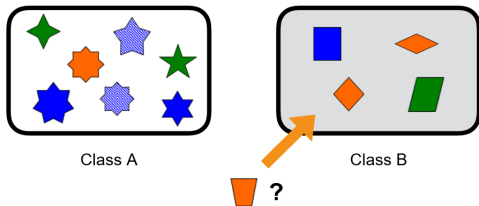


Figure 9: Classification result.

Clustering

- ▶ No labels
- ▶ Grouping based on similarity
- ▶ Unsupervised

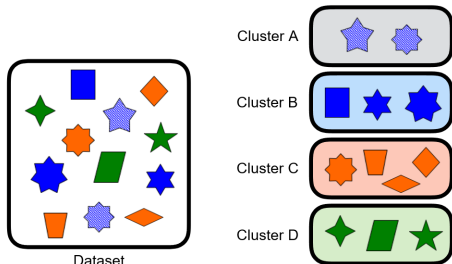


Figure 10: Clustering result.

[Source: CME 250, S. Wang, Stanford]

- ▶ **Centroid-Based Clustering:** Groups data points based on their proximity to a central point, such as K-means or K-medoids.
- ▶ **Hierarchical Clustering:** Builds a hierarchy of clusters using either agglomerative (bottom-up) or divisive (top-down) approaches.
- ▶ **Density-Based Clustering:** Grows clusters out of regions where points are packed closely together and labels sparse regions as noise, such as DBSCAN or OPTICS.
- ▶ **Model-Based Clustering:**
 - Each cluster is represented by a parametric distribution.
 - Dataset is a mixture of distributions.
 - Assumes a probabilistic model for the data and uses statistical methods to identify clusters, such as Gaussian Mixture Models (GMM).
- ▶ **Hard Clustering:**
 - Each data point is assigned exclusively to exactly one cluster.

- **Example algorithms:** K-means, Hierarchical clustering.
- **Interpretation:** No ambiguity; clusters are crisp and non-overlapping.

► **Soft/Fuzzy Clustering:**

- Each data point can belong to multiple clusters simultaneously with varying degrees of membership (probabilities or weights).
- **Example algorithms:** Gaussian Mixture Models (GMM), Fuzzy C-means.
- **Interpretation:** Reflects uncertainty or mixed membership; clusters can overlap.

Groups data into K clusters that satisfy two properties.

1. Each observation belongs to at least one of the K clusters.
2. Clusters are non-overlapping. No observation belongs to more than one cluster.

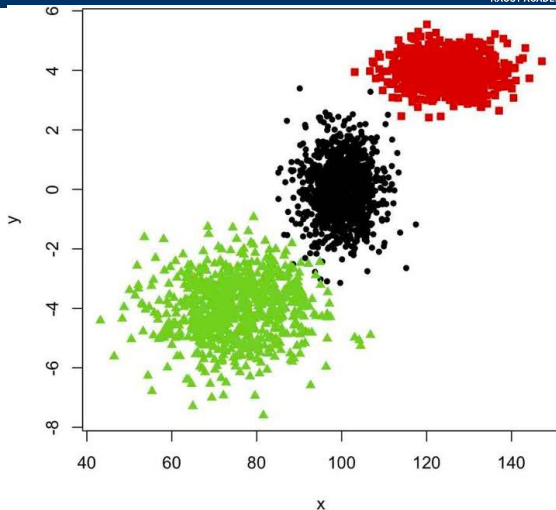


Figure 11: Clusters.

Clustering - K-means (cont.)

A good clustering is one for which the *within-cluster variation* is as small as possible.

Denote each cluster by C_k , and let $W(C_k)$ be a measure of the within-cluster variation.

K-means aims to solve the following optimization problem:

$$\underset{C_1, \dots, C_K}{\text{minimise}} \left\{ \sum_{k=1}^K W(C_k) \right\} \quad (1)$$

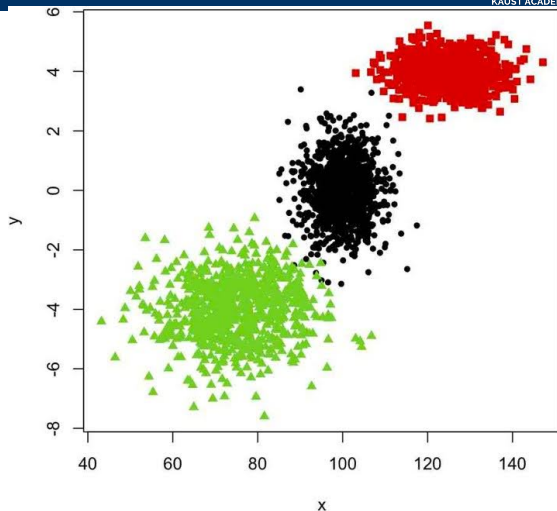


Figure 12: Clusters.

How to measure within-cluster variation?

The most common choice is squared Euclidean distance:

$$W(C_k) = \frac{1}{|C_k|} \sum_{i, i' \in C_k} \sum_{j=1}^p (x_{ij} - x_{i'j})^2 \quad (2)$$

where $|C_k|$ is the number of points in cluster C_k and x_{ij} is the j^{th} feature of the i^{th} point.

Which means overall we solve:

$$\underset{C_1, \dots, C_K}{\text{minimise}} \left\{ \sum_{k=1}^K \frac{1}{|C_k|} \sum_{i, i' \in C_k} \sum_{j=1}^p (x_{ij} - x_{i'j})^2 \right\} \quad (3)$$

- ▶ It turns out that this optimization problem is difficult to solve, as it is discrete and there are nearly K^n ways to split n samples into K clusters.
- ▶ In practice, use an iterative algorithm that finds a local minimum to this optimization.

Input: Dataset $D = \{x_1, x_2, \dots, x_n\}$, number of clusters k

Output: Cluster assignments for each data point

Initialization: Randomly initialize k cluster centroids or seeds;

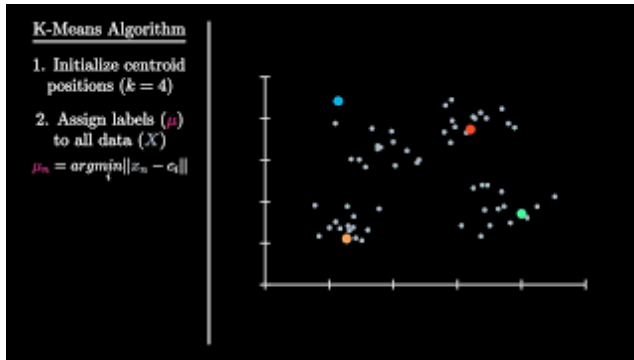
Repeat until convergence:

- ▶ **Assignment Step:** Assign each data point x_i to the nearest cluster based on a distance metric;
- ▶ **Update Step:** Recompute cluster centroids using current assignments;
- ▶ **Convergence Check:** Check if cluster assignments have changed or if centroids have stabilized;

Return: Final cluster assignments and centroids;

Algorithm: K-means Clustering Algorithm

Watch the K-means clustering algorithm in action:



[Video thumbnail: "K-Means Algorithm Visualization" (YouTube), linked above]

1. It can be shown that the value of the objective function will never increase at each iteration of k -means.
2. Since the algorithm finds local minima, however, it will result in different clusters with different initializations.

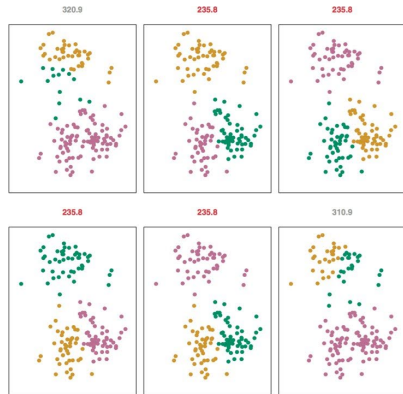


Figure 13: Different initializations of K-means. [ISL Fig. 10.7 (8th printing 2017), via CME 250, S. Wang, Stanford]

Pros

- ▶ Simple and easy to implement
- ▶ Scales to large datasets
- ▶ Works well with spherical clusters

Cons

- ▶ Sensitive to initial centroid placement and to small perturbations of the data
- ▶ Assumes spherical clusters
- ▶ Requires specifying the number of clusters K in advance
- ▶ Sensitive to outliers
- ▶ May converge to local minima
- ▶ Not suitable for non-convex shapes

Cluster based on distances between observations.

Represented as a tree hierarchy (*dendrogram*) rather than a partition of data.

Does not require committing to a choice of K .

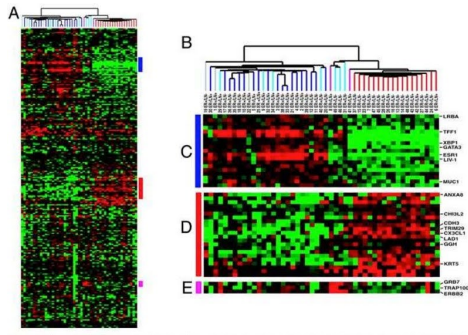


Figure 14: Sørbye, Therese, et al. (2003) "Repeated observation of breast tumor subtypes in independent gene expression data sets," PNAS.

- ▶ Each leaf in a dendrogram is a sample/ observation.
- ▶ As we move up the dendrogram, observations that are similar to each other begin to fuse into branches.
- ▶ Branches then fuse into bigger branches.
- ▶ Observations that fuse later (near the top of the tree, or root) are more different than observations that fuse earlier (near the leaves).

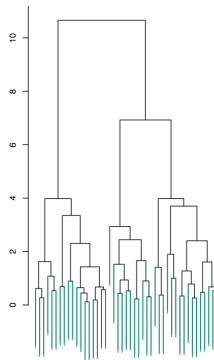


Figure 15: James, Witten, Hastie & Tibshirani, *An Introduction to Statistical Learning (ISL)*, 8th printing 2017

Note that the horizontal distance between observations on a dendrogram is not the appropriate assessment of observation similarity. Instead, look at vertical axis where branches are first fused.

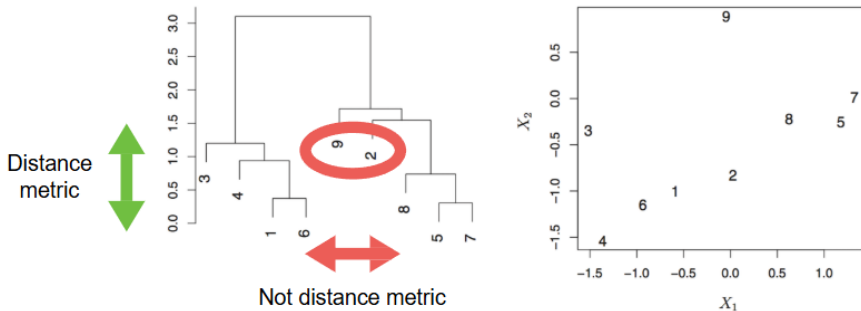


Figure 16: ISL (8th printing 2017)

Clusters are created by making a horizontal cut across the dendrogram. Clusters are the separate trees below the cut.

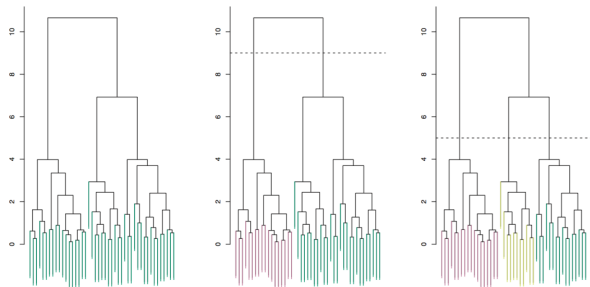


Figure 17: ISL (8th printing 2017)

Building a Dendrogram

A dendrogram is most commonly built using a bottom-up or agglomerative algorithm.

We start at the leaves and group observations until we reach the root containing the entire dataset.

Like in k -means, we need a measure of similarity. Again, the most common is Euclidean distance.

- ▶ Compute the distance between each pair of observations.
- ▶ Merge the two closest observations into a cluster.
- ▶ Compute the distance between the new cluster and all other observations.
- ▶ Repeat until all observations are in one cluster.
- ▶ The distance between clusters is computed using a linkage method.

Input: Dataset $D = \{x_1, x_2, \dots, x_n\}$

Output: Dendrogram representing the hierarchical structure of clusters

Initialization: Treat each data point as a separate cluster;

Compute distance matrix: Calculate pairwise distances between all clusters;

Repeat until only one cluster remains:

- ▶ Find the two closest clusters based on the distance matrix;
- ▶ Merge the two clusters into a new cluster;
- ▶ Update the distance matrix to reflect the new cluster;
- ▶ Recompute distances between the new cluster and all other clusters using a linkage method;

Return: Dendrogram representing the hierarchical structure of clusters;

Algorithm: Hierarchical Clustering Algorithm

w3schools: [Codes and Playground](#)

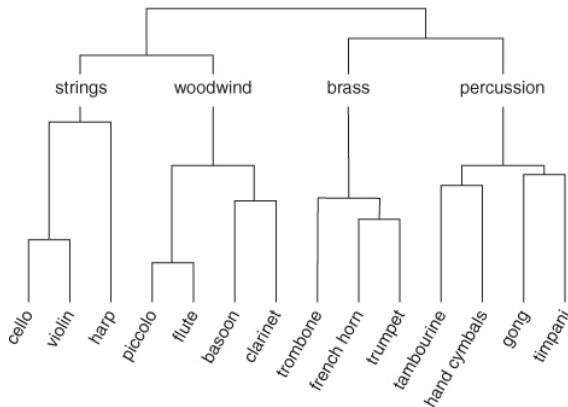


Figure 18: Dendrogram interpretation.

Distance between groups

It's easy to compute Euclidean distance between two observations. What is the distance or similarity between two groups or clusters of observations?

Linkage: defines the dissimilarity between two groups of observations. Most common types are *complete*, *average*, *single*, and *centroid*.

- ▶ **Single Linkage:** Distance between two clusters is the minimum distance between any two points in the clusters.
- ▶ **Complete Linkage:** Distance between two clusters is the maximum distance between any two points in the clusters.
- ▶ **Average Linkage:** Distance between two clusters is the average distance between all pairs of points in the clusters.
- ▶ **Centroid Linkage:** Distance between two clusters is the distance between their centroids.
- ▶ **Ward's Linkage:** Distance between two clusters is the increase in variance when the two clusters are merged.

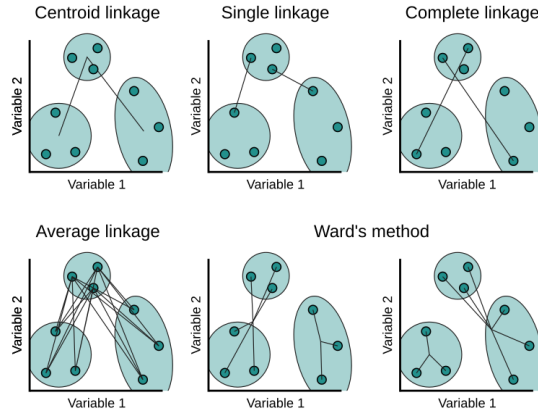


Figure 19: Linkage methods.

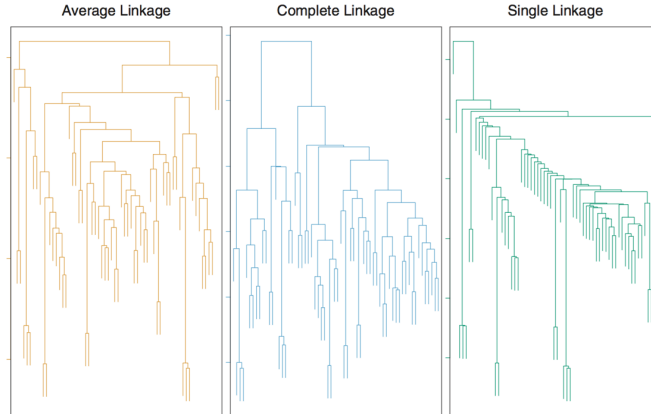


Figure 20: Dendrogram with different linkage types. [ISL Fig. 10.12 (8th printing 2017), via CME 250, S. Wang, Stanford]

Pros

- ▶ No need to specify the number of clusters K in advance.
- ▶ Dendrograms provide a visual representation of the clustering process.
- ▶ Can capture complex cluster shapes and relationships.

Cons

- ▶ Computationally expensive for large datasets.
- ▶ Must choose where to cut the dendrogram, and the cut height that gives the best clusters is not obvious.
- ▶ Sensitive to similarity measure and type of linkage used.
- ▶ Sensitive to noise and outliers.

- ▶ Clustering based on density (local cluster criterion), such as density-connected points or based on an explicitly constructed density function.
- ▶ **Major features:**
 - Discover clusters of arbitrary shape
 - Handle noise
 - One scan
 - Need density parameters

► Major algorithms:

- DBSCAN (Density-Based Spatial Clustering of Applications with Noise): Ester, et al. (KDD'96)
- OPTICS (Ordering Points to Identify the Clustering Structure): Ankerst, et al. (SIGMOD'99)
- HDBSCAN (Hierarchical Density-Based Spatial Clustering of Applications with Noise): Campello, et al. (ACM TKDD'15)
- DENCLUE (DENSITY-based CLUstEring): Hinneburg and Keim (KDD'98)
- CLIQUE (CLustering In QUEst): Agrawal, et al. (SIGMOD'98)

► **DBSCAN** (Density-Based Spatial Clustering of Applications with Noise):

- Density = number of points within a specified radius ϵ .
- A point is a core point if it has at least MinPts points within ϵ (including itself). These are points that are at the interior of a cluster.
- A border point has fewer than MinPts within ϵ , but is in the neighborhood of a core point
- A noise point is any point that is not a core point or a border point.
- Groups together points that are closely packed together, marking as outliers points that lie alone in low-density regions.
- Parameters:
 - ϵ : Maximum distance between two points for them to be considered as in the same neighborhood.
 - MinPts: Minimum number of points required to form a dense region.

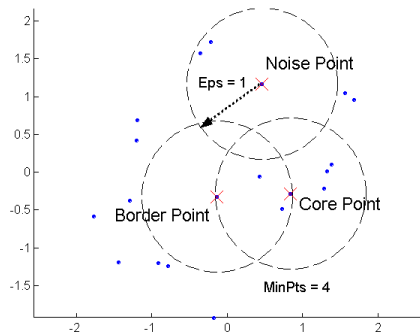


Figure 21: DBSCAN features: Core, Border, and Noise Points [Tan, Steinbach & Kumar, *Introduction to Data Mining*, via C. F. Eick, Machine Learning, Univ. of Houston]

Input: Set of points P , distance threshold ε , minimum number of points $minPts$

Output: A set of clusters

Construct a directed graph $G = (V, E)$ where each node in V corresponds to a point in P ;

foreach point $c \in P$ **do**

if c is a core point (i.e., $|\mathcal{N}_\varepsilon(c)| \geq minPts$) **then**

foreach point $p \in \mathcal{N}_\varepsilon(c)$ **do**

 Add a directed edge ($c \rightarrow p$) to E ;

end

end

end

$N \leftarrow V$;

while there exists a core point $c \in N$ **do**

 Let X be the set of nodes reachable from c via directed edges in G ;

 Form a cluster $C = X \cup \{c\}$;

 Remove all nodes in C from N ;

end

Algorithm: Graph-Based DBSCAN Clustering



Figure 22: Original Points

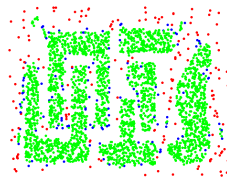


Figure 23: Point types: **Core**, **Border**, and **Noise** Points [Tan, Steinbach & Kumar, *Introduction to Data Mining*]

$Eps = 10$, $MinPts = 4$



Figure 24: Original Points

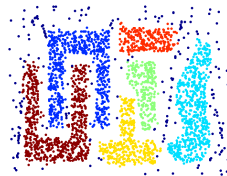


Figure 25: Clusters identified by DBSCAN

DBSCAN works well when:

- ▶ Clusters are of varying shapes and sizes.
- ▶ There is a clear distinction between dense and sparse regions.
- ▶ The data contains noise or outliers.

[Tan, Steinbach & Kumar, *Introduction to Data Mining*, via C. F. Eick, Machine Learning, Univ. of Houston]

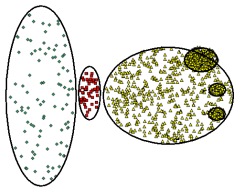


Figure 26: Original Points

DBSCAN does not work well when:

- ▶ Clusters are of varying densities.
- ▶ The data has varying scales or dimensions.
- ▶ There are overlapping clusters.
- ▶ The choice of ϵ and MinPts is not suitable for the data distribution.

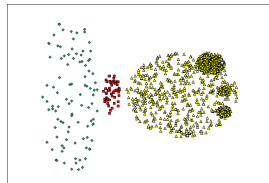


Figure 27: MinPts=4, $\epsilon=9.75$

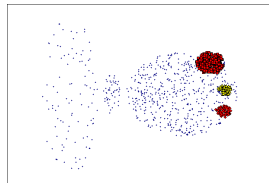


Figure 28: MinPts=4, $\epsilon=9.92$

[Tan, Steinbach & Kumar, *Introduction to Data Mining*, via C. F. Eick, Machine Learning, Univ. of Houston]

- ▶ A common approach to determine ϵ is to use a k-distance graph:
 - For each point, compute the distance to its k-th nearest neighbor.
 - Plot these distances in ascending order.
 - Look for a “knee” in the plot, which indicates a suitable value for ϵ .
- ▶ MinPts is often set based on domain knowledge or heuristics:
 - A common rule of thumb is to set MinPts to at least the dimensionality of the data plus one (e.g., for 2D data, MinPts = 3).
 - Higher values of MinPts can lead to fewer clusters and more noise points.

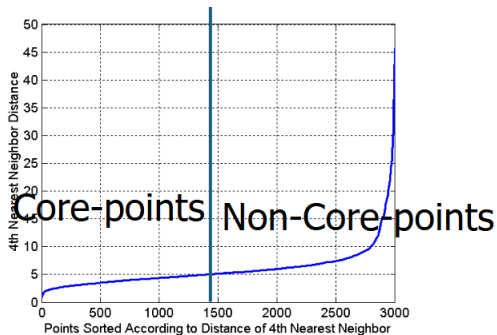


Figure 29: Sorted 4-nearest-neighbour distance plot (MinPts=4); the knee gives ϵ [Tan, Steinbach & Kumar, *Introduction to Data Mining*; core/non-core annotation from C. F. Eick, Machine Learning, Univ. of Houston]

► Complexity:

- The time complexity of DBSCAN is $O(n \log n)$ for spatial data structures like KD-trees or R-trees, where n is the number of points.
- Without spatial indexing, the complexity can degrade to $O(n^2)$.
- Space complexity is $O(n)$, as it needs to store the points and their cluster assignments.

► Limitations:

- Sensitive to the choice of ϵ and MinPts parameters.
- Struggles with clusters of varying densities.
- Not suitable for high-dimensional data due to the curse of dimensionality.
- Cannot handle clusters that are not well-separated in terms of density.

► Good:

- Can discover clusters of arbitrary shape.
- Effectively handles noise and outliers.
- Requires only one scan of the data.
- Does not require prior knowledge of the number of clusters.

► Bad:

- Does not work well in high-dimensional datasets.
- Parameter selection is tricky.
- Has problems identifying clusters of varying densities (→ SNN (Shared Nearest Neighbour) algorithm).
- Density estimation is simplistic (→ does not create a real density function, but rather a graph of density-connected points).

```
current_cluster_label  $\leftarrow$  1
for all core points do
  if the core point has no cluster label then
    current_cluster_label  $\leftarrow$  current_cluster_label + 1
    Label the current core point with cluster label current_cluster_label
  end if
  for all points in the Eps-neighborhood, except  $i^{th}$  the point itself do
    if the point does not have a cluster label then
      Label the point with cluster label current_cluster_label
    end if
  end for
end for
```

Figure 30: DBSCAN Algorithm Steps [Tan, Steinbach & Kumar, *Introduction to Data Mining*, via C. F. Eick, Machine Learning, Univ. of Houston]

Where are we heading?

Gene	Description	Cell 1	Cell 2	Cell 3	Cell 4	Cell 5
Inpp5d	inositol polyphosphate-5-phosphatase D	7.00	5.45	5.89	6.03	5.75
Aim2	absent in melanoma 2	3.01	4.37	4.59	4.38	4.18
Gldn	gliomedin	3.48	3.63	4.61	4.70	4.74
Frem2	Fras1 related extracellular matrix protein 2	4.75	4.66	3.46	3.74	3.45
Rps3a1	ribosomal protein S3A1	6.10	7.23	7.44	7.36	7.34
Slc38a3	solute carrier family 38, member 3	1.90	3.16	3.52	3.61	3.19
Mt1	metallothionein 1	5.07	6.49	6.46	6.04	6.05
C1s1	complement component 1, s subcomponent 1	2.74	3.02	3.86	4.10	4.10
Cds1	CDP-diacylglycerol synthase 1	4.55	4.22	3.80	3.16	3.12
Ifi44	interferon-induced protein 44	4.82	4.52	3.87	3.42	3.59
Lefty2	left-right determination factor 2	6.95	6.28	5.88	5.60	5.61
Fmr1nb	fragile X mental retardation 1 neighbor	4.28	2.78	3.10	3.25	2.57
Tagln	transgelin	7.93	7.91	7.20	7.02	6.68

Figure 31: Gene expression matrix (genes $\times$ cells)

- ▶ Each dot is a cell.
- ▶ Groups of dots are similar cells.
- ▶ Separation of groups could be interesting biology.

[Source: S. Andrews, "Dimension Reduction", Babraham Bioinformatics]

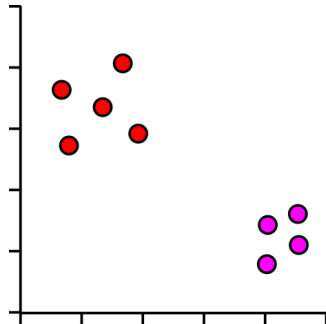
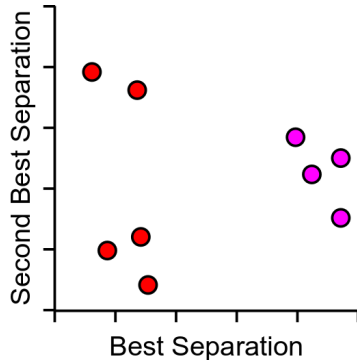
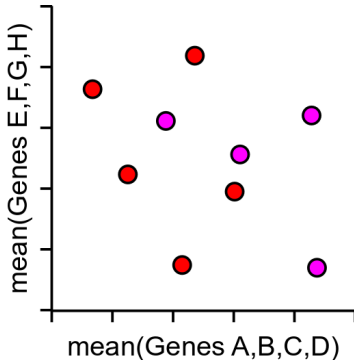
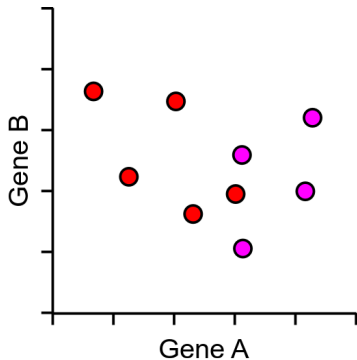


Figure 32: Cells projected to 2D, coloured by group

Too much data!

- ▶ 5000 cells and 2500 measured genes
- ▶ Realistically only 2 dimensions we can plot (x, y)



[Source: S. Andrews, "Dimension Reduction", Babraham Bioinformatics]

- ▶ Principal Component Analysis (PCA) is a method for optimally summarizing large, multi-dimensional datasets.
- ▶ PCA identifies a smaller number of dimensions (ideally 2) that retain most of the useful information present in the original data.
- ▶ The technique constructs a set of new variables, called Principal Components (PCs), which are linear combinations of the original variables.
- ▶ Each Principal Component represents a specific weighted sum of the original features. For example:

$$PC = (10 \times \text{GeneA}) + (3 \times \text{GeneB}) + (-4 \times \text{GeneC}) + (-20 \times \text{GeneD}) + \dots$$

- ▶ By projecting the data onto these principal components, PCA reduces dimensionality while preserving as much variance (information) as possible.

- ▶ Principal Component Analysis (PCA) is a method for optimally summarizing large, multi-dimensional datasets.
- ▶ PCA identifies a smaller number of dimensions (ideally 2) that retain most of the useful information present in the original data.
- ▶ The technique constructs a set of new variables, called Principal Components (PCs), which are linear combinations of the original variables.
- ▶ Each Principal Component represents a specific weighted sum of the original features. For example:

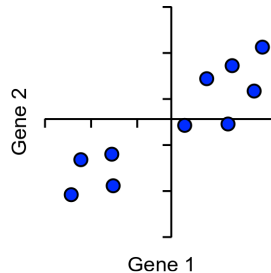
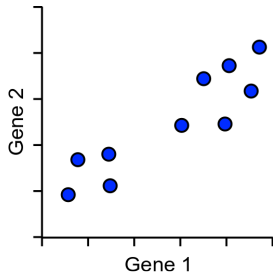
$$PC = (10 \times \text{GeneA}) + (3 \times \text{GeneB}) + (-4 \times \text{GeneC}) + (-20 \times \text{GeneD}) + \dots$$

- ▶ By projecting the data onto these principal components, PCA reduces dimensionality while preserving as much variance (information) as possible.

Simple example using 2 genes and 10 cells

How does PCA work?

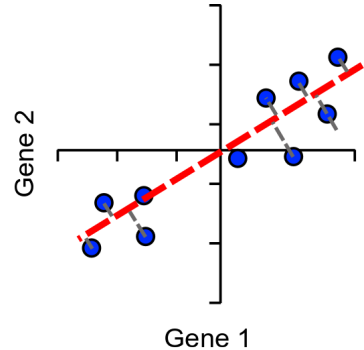
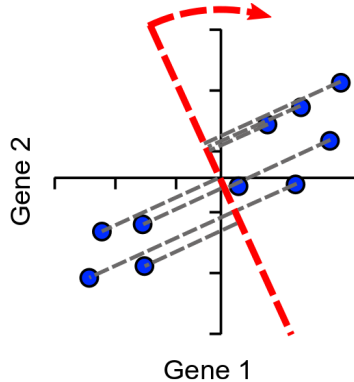
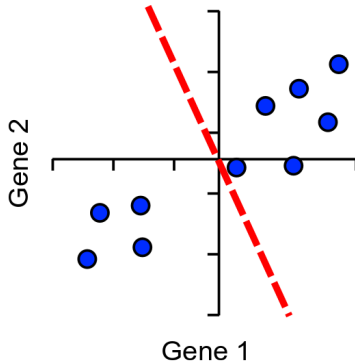
Simple example using 2 genes and 10 cells



[Source: S. Andrews, "Dimension Reduction", Babraham Bioinformatics]

How does PCA work? (cont.)

Find line of best fit, passing through the origin



[Source: S. Andrews, "Dimension Reduction", Babraham Bioinformatics]

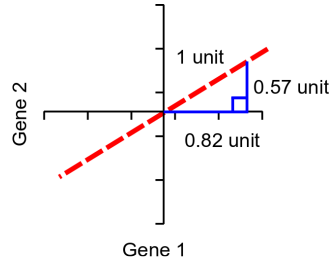
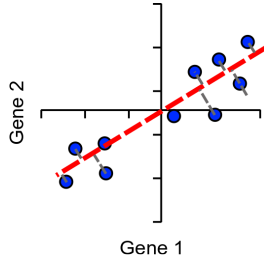
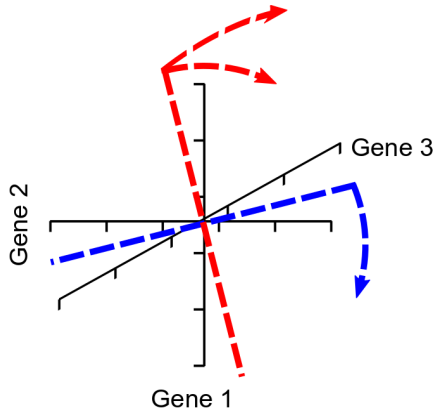


Figure 33: **Eigenvector** or Singular Vector [Source: S. Andrews, "Dimension Reduction", Babraham Bioinformatics]

- ▶ **Loadings** represent the contribution of each gene to the principal component.
- ▶ For this example using 2 genes and 10 cells:
 - Gene1 loading: 0.82
 - Gene2 loading: 0.57

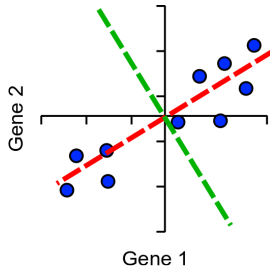
- ▶ A higher loading means the gene has a greater influence on the principal component.



- ▶ The same idea extends to datasets with more dimensions (n genes).
- ▶ The first principal component (PC1) finds the direction of maximum variance, rotating in $(n - 1)$ dimensions.
- ▶ The next principal component (PC2) is perpendicular to PC1 and rotates in the remaining $(n - 2)$ dimensions.
- ▶ Each subsequent principal component is always perpendicular to the previous ones, capturing the next largest variance.
- ▶ The last principal component is the only remaining perpendicular direction.
- ▶ The number of principal components is at most $\min(\text{number of samples} - 1, \text{number of features})$.

[Source: S. Andrews, "Dimension Reduction", Babraham Bioinformatics]

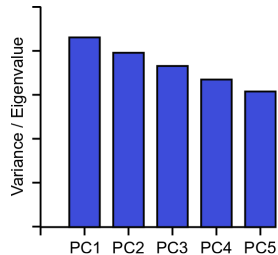
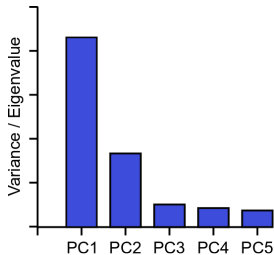
- ▶ Each principal component (PC) explains a certain proportion of the total variance in the data. Together, all PCs account for 100% of the variance.
- ▶ PC1 always explains the largest amount of variance.
- ▶ PC2 explains the next largest amount, and so on for subsequent PCs.
- ▶ Since we typically plot only 2 dimensions (PC1 and PC2), it's important to check how much of the total variance these two PCs explain.
- ▶ **How do we calculate this?**



- ▶ Project each data point onto the principal component (PC).
- ▶ Calculate the distance of each projected point to the origin.
- ▶ Compute the sum of squared distances (SSD) from the origin.
- ▶ This SSD is a measure of variance, called the *eigenvalue*.
- ▶ Divide the SSD by (number of points – 1) to obtain the actual variance explained by the PC.

[Source: S. Andrews, "Dimension Reduction", Babraham Bioinformatics]

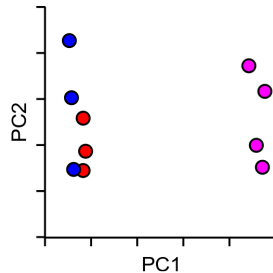
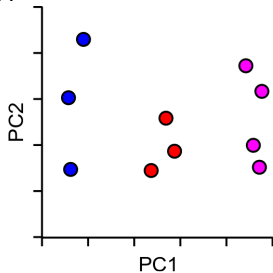
Explaining Variance (cont.)



[Source: S. Andrews, "Dimension Reduction", Babraham Bioinformatics]

So PCA is great then?

Kind of...

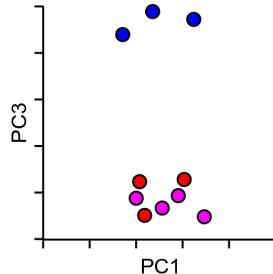
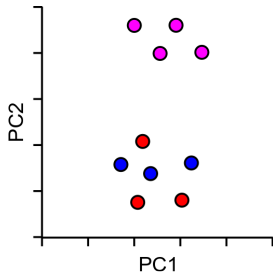


Non-linear separation of values

[Source: S. Andrews, "Dimension Reduction", Babraham Bioinformatics]

So PCA is great then? (cont.)

Kind of...



Not optimised for 2-dimensions

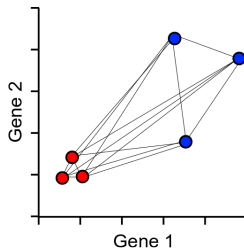
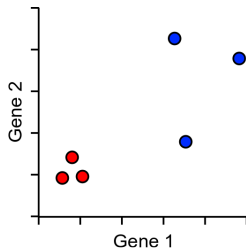
[Source: S. Andrews, "Dimension Reduction", Babraham Bioinformatics]

- ▶ t-Distributed Stochastic Neighbour Embedding
- ▶ Aims to solve the problems of PCA
- ▶ Non-linear scaling to represent changes at different levels
- ▶ Optimal separation in 2-dimensions

[t-SNE, van der Maaten & Hinton, JMLR 9, 2008; builds on SNE, Hinton & Roweis, NIPS 2002]

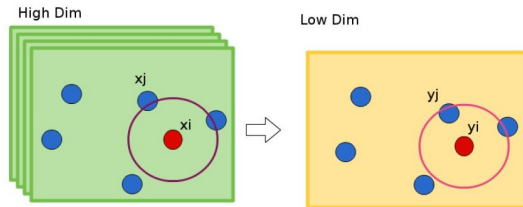
tSNE: How does it work?

- ▶ Based around all-vs-all table of pairwise cell to cell distances
- ▶ Each cell is represented as a point in a high-dimensional space
- ▶ The pairwise distances are converted into probabilities



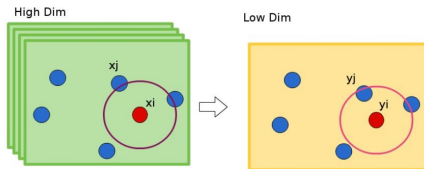
	0	10	10	295	158	153
	9	0	1	217	227	213
	1	8	0	154	225	238
	205	189	260	0	23	45
	248	227	246	44	0	54
	233	176	184	41	36	0

[Source: S. Andrews, "Dimension Reduction", Babraham Bioinformatics]



[Source: Z. Fataliyev, "Dimensionality reduction with t-SNE", Seoul AI]

Measure pairwise similarities between high-dimensional and low-dimensional objects



$$p_{j|i} = \frac{\exp(-\|x_i - x_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|x_i - x_k\|^2 / 2\sigma_i^2)}$$

[Source: Z. Fataliyev, "Dimensionality reduction with t-SNE", Seoul AI]

Converting the high-dimensional Euclidean distances into conditional probabilities that represent similarities

- Similarity of datapoints in High Dimension

$$p_{j|i} = \frac{\exp(-||x_i - x_j||^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-||x_i - x_k||^2 / 2\sigma_i^2)}$$

- Similarity of datapoints in Low Dimension

$$q_{j|i} = \frac{\exp(-||y_i - y_j||^2)}{\sum_{k \neq i} \exp(-||y_i - y_k||^2)}$$

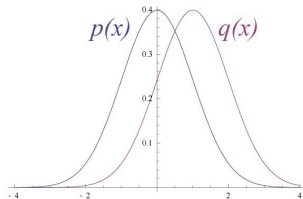
- Cost function

$$C = \sum_i KL(P_i || Q_i) = \sum_i \sum_j p_{j|i} \log \frac{p_{j|i}}{q_{j|i}}$$

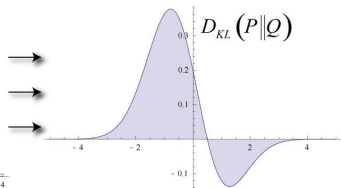
[Source: Z. Fataliyev, "Dimensionality reduction with t-SNE", Seoul AI]

Measures how one probability distribution diverges from another; it is asymmetric.

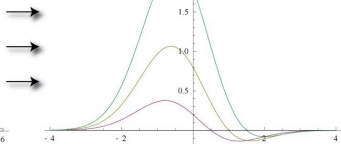
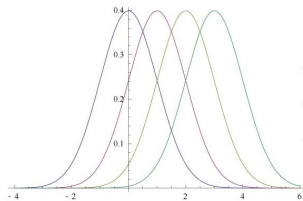
$$D_{\text{KL}}(P\|Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)},$$



Original Gaussian PDF's



KL Area to be Integrated



[Source: Z. Fataliyev, "Dimensionality reduction with t-SNE", Seoul AI]

Gradient has a surprisingly simple form

$$\frac{\partial C}{\partial y_i} = \sum_{j \neq i} (p_{j|i} - q_{j|i} + p_{i|j} - q_{i|j})(y_i - y_j)$$

The gradient update with momentum term is given by

$$Y^{(t)} = Y^{(t-1)} + \eta \frac{\partial C}{\partial y_i} + \beta(t)(Y^{(t-1)} - Y^{(t-2)})$$

[Source: Z. Fataliyev, "Dimensionality reduction with t-SNE", Seoul AI]

- ▶ The result of running the SNE algorithm on 3000 256-dimensional grayscale images of handwritten digits.
- ▶ The classes are quite well separated even though SNE had no information about class labels.
- ▶ Furthermore, within each class, properties like orientation, skew, and stroke thickness tend to vary smoothly across the space.

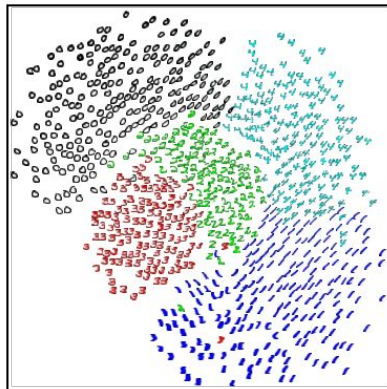


Figure 34: [Hinton & Roweis, “Stochastic Neighbor Embedding”, NIPS 2002, Fig. 1]

Such that $p_{ij} = p_{ji}$, $q_{ij} = q_{ji}$, the main advantage is simplifying the gradient

$$\frac{\partial C}{\partial y_i} = 2 \sum_j (p_{ij} - q_{ij})(y_i - y_j)$$

However, in practice we symmetrize (or average) the conditionals

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2N}$$

Set the bandwidth σ_i such that the conditional has a fixed perplexity (effective number of neighbors) $\text{Perp}(P_i) = 2^{H(P_i)}$, typical value is about 5 to 50

[Source: Z. Fataliyev, "Dimensionality reduction with t-SNE", Seoul AI]

Use heavier tail distribution than Gaussian in low-dim space, we choose

$$q_{ij} \propto (1 + \|y_i - y_j\|^2)^{-1}$$

Then the gradient could be

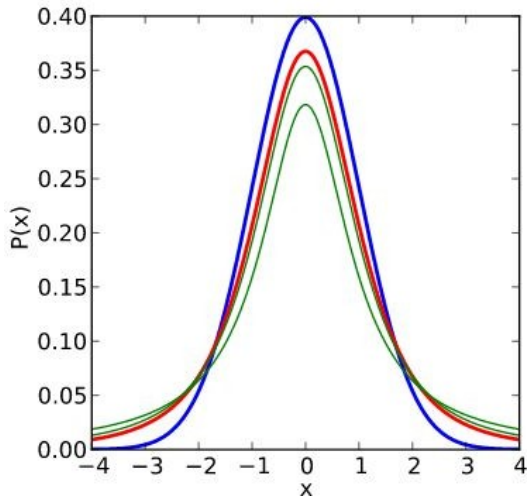
$$\frac{\partial C}{\partial y_i} = 4 \sum_{j \neq i} (p_{ij} - q_{ij}) (1 + \|y_i - y_j\|^2)^{-1} (y_i - y_j)$$

[Source: Z. Fataliyev, "Dimensionality reduction with t-SNE", Seoul AI]

- Why do we define map similarities as:

$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq l} (1 + \|y_k - y_l\|^2)^{-1}}$$

- Suppose data is intrinsically high dimensional
- We try to model the local structure of this data in the map
- Result: Dissimilar points have to be modeled as too far apart in the map!



[Source: Z. Fataliyev, "Dimensionality reduction with t-SNE", Seoul AI]

- Similarity of datapoints in High Dimension

$$p_{ij} = \frac{\exp(-||x_i - x_j||^2/2\sigma^2)}{\sum_{k \neq i} \exp(-||x_i - x_k||^2/2\sigma^2)}$$

- Similarity of datapoints in Low Dimension

$$q_{ij} = \frac{(1 + ||y_i - y_j||^2)^{-1}}{\sum_{k \neq i} (1 + ||y_k - y_i||^2)^{-1}}$$

[Source: Z. Fataliyev, "Dimensionality reduction with t-SNE", Seoul AI]

- Cost function

$$C = KL(P||Q) = \sum_i \sum_j p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

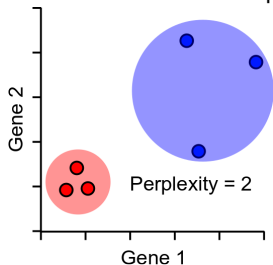
- Large p_{ij} modeled by small q_{ij} : Large penalty
- Small p_{ij} modeled by large q_{ij} : Small penalty
- t-SNE mainly preserves local similarity structure of the data

- Gradient

$$\frac{\partial C}{\partial y_i} = 4 \sum_{j \neq i} (p_{ij} - q_{ij}) (1 + \|y_i - y_j\|^2)^{-1} (y_i - y_j)$$

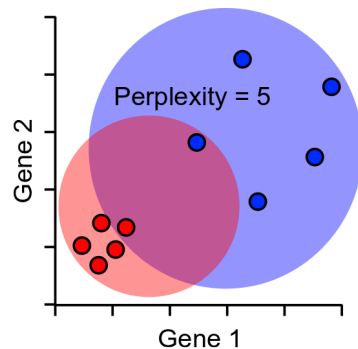
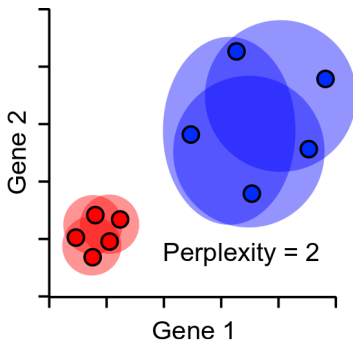
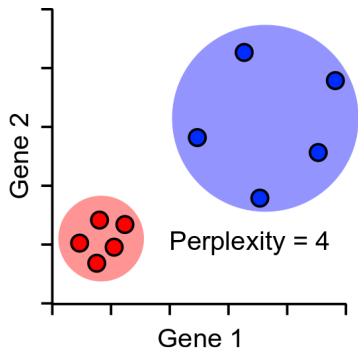
[Source: Z. Fatahian, "Dimensionality reduction with t-SNE", Seoul AI]

- ▶ Perplexity = effective number of nearest neighbours each point is compared against
- ▶ Distances scaled relative to perplexity neighbours



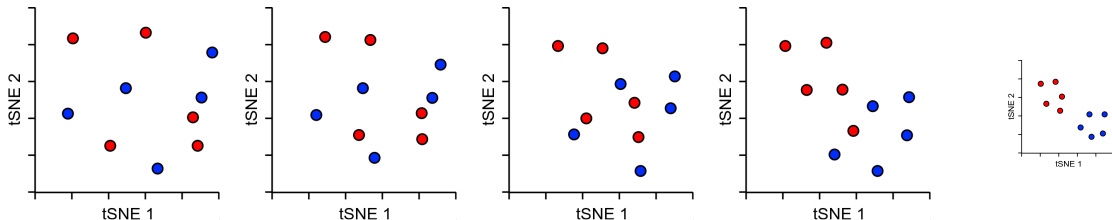
	0	4	6	586	657	836
	4	0	4	815	527	776
	9	3	0	752	656	732
	31	28	29	0	4	7
	31	24	25	4	0	7
	40	37	32	8	8	0

[Source: S. Andrews, "Dimension Reduction", Babraham Bioinformatics]



[Source: S. Andrews, "Dimension Reduction", Babraham Bioinformatics]

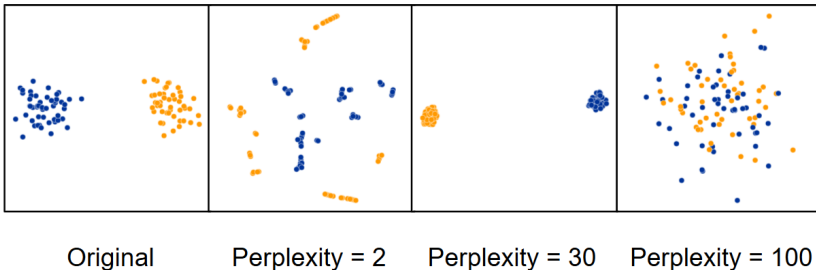
- ▶ Randomly scatter all points within the space (normally 2D)
- ▶ Start a simulation
 - Aim is to make the point distances match the distance matrix
 - Shuffle points based on how well they match
 - Stop after fixed number of iterations, or
 - Stop after distances have converged



- ▶ X and Y don't mean anything (unlike PCA)
- ▶ Distance doesn't mean anything (unlike PCA)
- ▶ Close proximity is highly informative
- ▶ Distant proximity isn't very interesting
- ▶ Can't rationalise distances, or add in more data

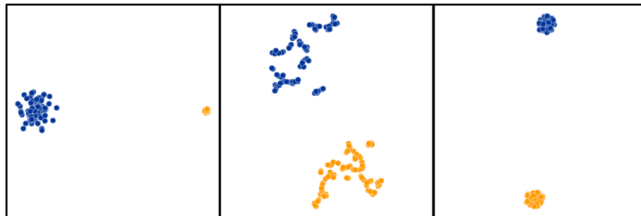
[Source: S. Andrews, "Dimension Reduction", Babraham Bioinformatics]

Perplexity Settings Matter



[Wattenberg, Viégas & Johnson, "How to Use t-SNE Effectively", Distill 2016, via S. Andrews, "Dimension Reduction", Babraham Bioinformatics]

Cluster Sizes are Meaningless



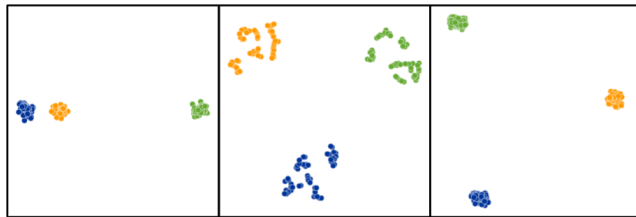
Original

Perplexity = 5

Perplexity = 50

[Wattenberg, Viégas & Johnson, "How to Use t-SNE Effectively", Distill 2016, via S. Andrews, "Dimension Reduction", Babraham Bioinformatics]

Distances between clusters can't be trusted



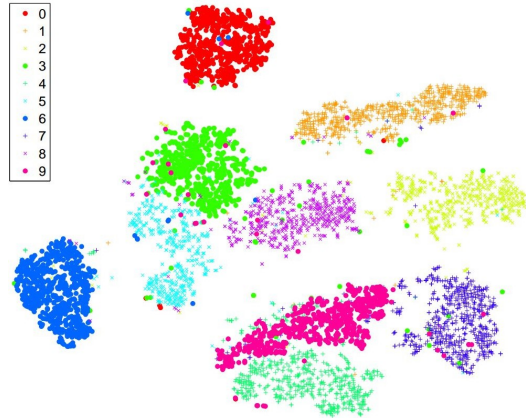
Original

Perplexity = 5

Perplexity = 30

[Wattenberg, Viégas & Johnson, "How to Use t-SNE Effectively", Distill 2016, via S. Andrews, "Dimension Reduction", Babraham Bioinformatics]

tSNE: Results on MNIST

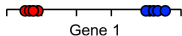


[Source: Z. Fatahiyev, "Dimensionality reduction with t-SNE", Seoul AI]

So tSNE is great then?

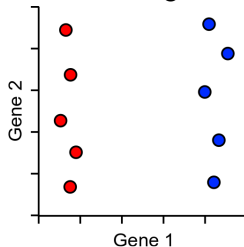
Kind of...

Imagine a dataset with only one super informative gene



Distance within cluster = low

Distance between clusters =
high



Distance within cluster =
higher

Distance between clusters =
lower

- ▶ First - 3 genes
- ▶ Second - 3,000 genes
- ▶ Everything is the same distance from everything

[Source: S. Andrews, "Dimension Reduction", Babraham Bioinformatics]

So there is no clear solution?

PCA:

- ▶ Requires more than 2 dimensions
- ▶ Expects linear relationships

tSNE:

- ▶ Can't cope with noisy data
- ▶ Loses the ability to cluster

So there is no clear solution? (cont.)

PCA:

- ▶ Requires more than 2 dimensions
- ▶ Expects linear relationships

tSNE:

- ▶ Can't cope with noisy data
- ▶ Loses the ability to cluster

Answer: Combine the two methods, get the best of both worlds!

PCA:

- ▶ Good at extracting signal from noise
- ▶ Extracts informative dimensions

tSNE:

- ▶ Can reduce to 2D well
- ▶ Can cope with non-linear scaling

This is what many pipelines do in their default analysis!

So PCA + tSNE is great then?

Kind of...

- ▶ tSNE is slow; this is its biggest drawback.
 - tSNE does not scale well to large datasets (10,000+ samples).
- ▶ tSNE provides reliable information only about the closest neighbors; large distance information is mostly irrelevant.

- ▶ UMAP is a replacement for t-SNE, designed to fulfill the same role in dimensionality reduction.
- ▶ It is conceptually very similar to t-SNE, but introduces several relevant (and somewhat technical) changes.
- ▶ In practice, UMAP offers several advantages:
 - UMAP is significantly faster than t-SNE.
 - It can preserve more global structure compared to t-SNE.¹
 - UMAP can operate directly on raw data without requiring PCA preprocessing.²
 - It allows new data points to be added to an existing projection, enabling incremental updates.

[UMAP, McInnes, Healy & Melville, 2018]

¹Preservation of global structure may depend on the dataset and parameter choices.

²Although PCA preprocessing can still be beneficial in some cases.

- ▶ Unlike t-SNE, which uses a single perplexity parameter, UMAP introduces two key parameters:
 - **Number of Nearest Neighbours:** Specifies how many nearest neighbours are considered for each point. This is conceptually similar to perplexity in t-SNE and controls the balance between local and global structure in the embedding.
 - **Minimum Distance:** Determines how closely UMAP packs points together in the low-dimensional space. Lower values allow points to be closer, resulting in more compact clusters, while higher values spread points apart.
- ▶ The **nearest neighbours** parameter influences the trade-off between preserving local versus global structure in the data.
- ▶ The **minimum distance** parameter affects the density and compactness of clusters in the resulting visualization.
- ▶ Together, these parameters provide more flexibility and control over the embedding compared to t-SNE's single perplexity value.

Structure preservation: mostly in the 2D projection scoring

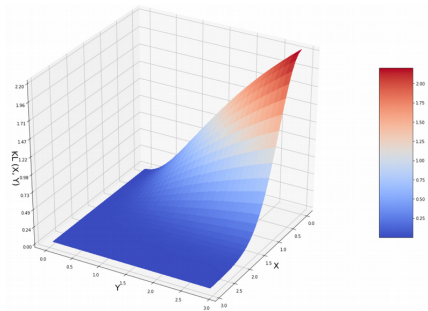


Figure 35: t-SNE

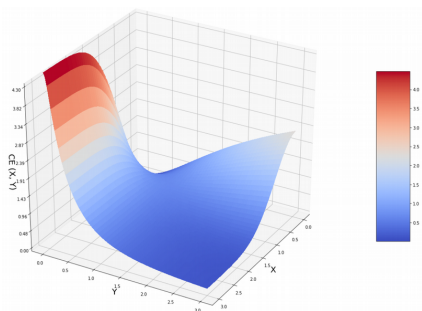


Figure 36: UMAP

Scoring
(penalty
)
value

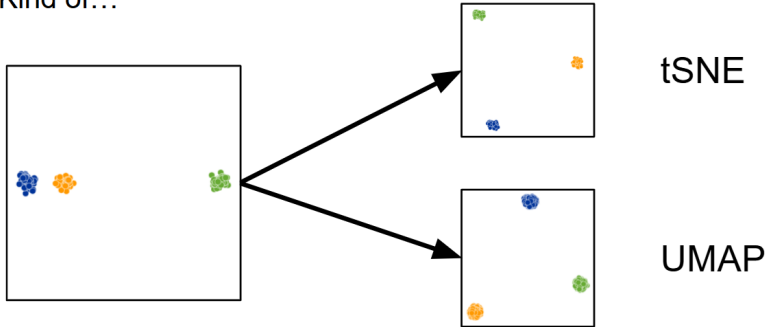
Distance in
projected
data

Distance in
original
data

[Source: S. Andrews, "Dimension Reduction", Babraham Bioinformatics]

So UMAP is great then?

Kind of...



[Source: S. Andrews, "Dimension Reduction", Babraham Bioinformatics]

So UMAP is all hype then?

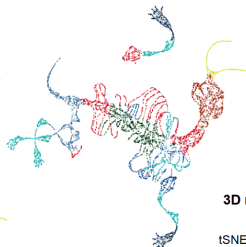
No, it really does better for some datasets...

2D t-SNE projection



<https://pair-code.github.io/understanding-umap/>

2D UMAP projection



3D mammoth skeleton projected into 2D

tSNE: Perplexity 2000 2h 5min

UMAP: Neigh 200, mindist 0.25, 3min

[Mammoth projections from <https://pair-code.github.io/understanding-umap/>, via S. Andrews, "Dimension Reduction", Babraham Bioinformatics]

► Filter the data:

- Remove poorly behaving cells to ensure data quality.
- Select only expressed genes for further analysis.
- Identify and retain variable genes to focus on informative features.

► Apply PCA (Principal Component Analysis):

- Extract the most interesting and informative signals from the data.
- Select the top principal components (PCs) to reduce dimensionality, but do not reduce directly to 2 dimensions.

► Run t-SNE or UMAP:

- Calculate distances between samples using the PCA projections.
- Scale these distances appropriately.
- Project the data into 2 dimensions for visualization using t-SNE or UMAP.

So PCA + UMAP is great then?

- ▶ Kind of... as long as you only have one dataset.
 - In 10X experiments, each library is considered a **batch**.
 - Over time or distance, additional biases can be introduced.
 - These biases make direct comparisons between datasets challenging.
 - To enable meaningful comparisons, it is necessary to align the datasets and correct for batch effects.

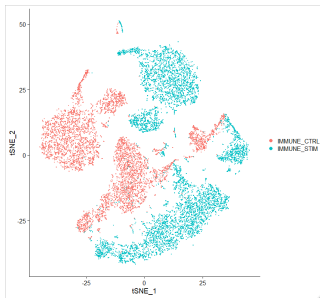


Figure 37: UMAP [Source: S. Andrews, "Dimension Reduction", Babraham Bioinformatics]

- ▶ Sherrie Wang, CME 250: Introduction to Machine Learning, Stanford University.
<https://web.stanford.edu/class/cme250/>
- ▶ Volodymyr Kuleshov, CS 5785: Applied Machine Learning, Cornell University.
<https://kuleshov-group.github.io/aml-website/>
- ▶ Aykut Erdem, COMP547: Deep Unsupervised Learning, Koç University.
<https://aykuterdem.github.io/classes/comp547.s24/>
- ▶ Christoph F. Eick, Machine Learning, University of Houston.
<http://www2.cs.uh.edu/~ceick/ML/Topic9.ppt>
- ▶ Zaur Fataliyev, Seoul AI. <https://seoulai.com/presentations/t-SNE.pdf>
- ▶ Simon Andrews, "Dimensionality Reduction", Babraham Bioinformatics.
<https://www.bioinformatics.babraham.ac.uk/training/10XRNASeq/Dimension%20Reduction.pptx>

Primary sources for the methods covered:

- ▶ James, Witten, Hastie & Tibshirani, *An Introduction to Statistical Learning* (ISL), Springer, 8th printing 2017.
- ▶ Tan, Steinbach & Kumar, *Introduction to Data Mining*, Addison-Wesley.
- ▶ Ester, Kriegel, Sander & Xu, "A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise", KDD 1996.
- ▶ Hinton & Roweis, "Stochastic Neighbor Embedding", NIPS 2002.
- ▶ van der Maaten & Hinton, "Visualizing Data using t-SNE", JMLR 9:2579–2605, 2008.
- ▶ McInnes, Healy & Melville, "UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction", <https://arxiv.org/abs/1802.03426>, 2018.
- ▶ Wattenberg, Viégas & Johnson, "How to Use t-SNE Effectively", *Distill*, 2016.
<https://distill.pub/2016/misread-tsne/>
- ▶ "Understanding UMAP", Google PAIR. <https://pair-code.github.io/understanding-umap/>